# Discriminative Project Final Report

IE 7615 Deep Learning and Neural Networks

*Sam Yuen*

*Ragul Narayanan Magesh*

*Nithin Arumugam Raja*

*Carina Davis*

# GitHub

# Dataset Overview

The training dataset consisted of 224x224 JPEG images of 39 unique objects, each of them having 100 samples each. This totaled to about 4000 photos total for the training set. The photos of each objects were taken from various angles, backgrounds and lighting conditions to reduce redundancy.

Some training images for a handful of objects were useless, not enough, or not properly compressed, which added a little bit of skepticism in the beginning on how well the model would detect certain objects. One student mistakenly uploaded a photo of someone taking photos of something. One student uploaded photos of an apple that were compressed at its width so the apple looked unrealistically ovular in many of its samples. One student uploaded photos of completely different lotion bottles so none of those bottles had 100 samples each. One student had photos of their water bottle that weren't in focus when the shutter button was pressed. Overall, the training data wasn't perfect.

# Annotation Process

To prepare the dataset for the YOLOv8 model, we used RoboFlow to annotate our training images, turning them into a structured dataset. Each image was annotated with vector-based bounding boxes to help the model detect and differentiate the objects in various angles, backgrounds, and lighting conditions. With precise labeling margins and class assignments for each object, we were able to minimize localization errors and background noise. After hours of annotating, the data was exported into PyTorch format. This converted our labels into the normalized coordinate system the YOLOv8 model needed for training.

# Performance

Throughout training, the model was very quickly converging with high stability which led to great performance. The final model had a mean average precision score of 0.964 for all 39 objects. It also had a balanced performance with its precision of 0.946 and a recall of 0.957. After tinkering with the F1 optimization, the model achieved a peak F1 score of 0.95 with a confident threshold of 0.68. The validation metrics at the end of the training process showed a box loss of 0.403 and a class loss of 0.308.

**Charts and Visualizations:**

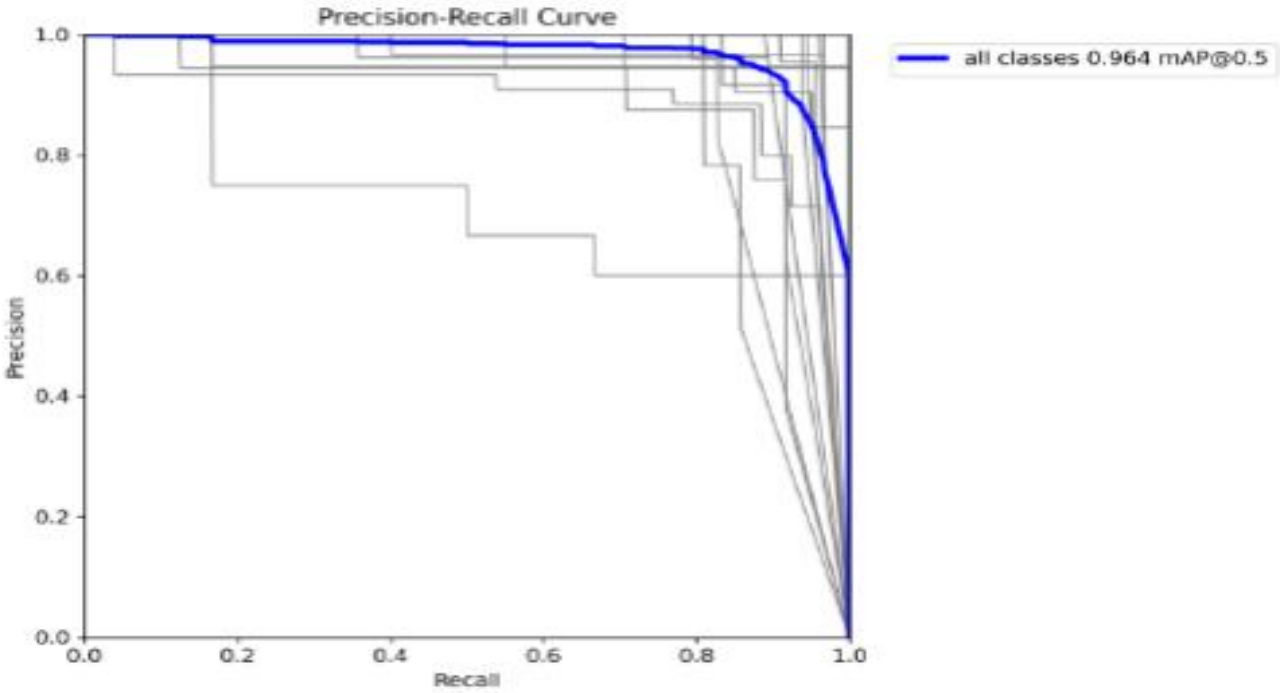



Figure 1: Precision-Recall Curve (mAP@0.5: 0.964).

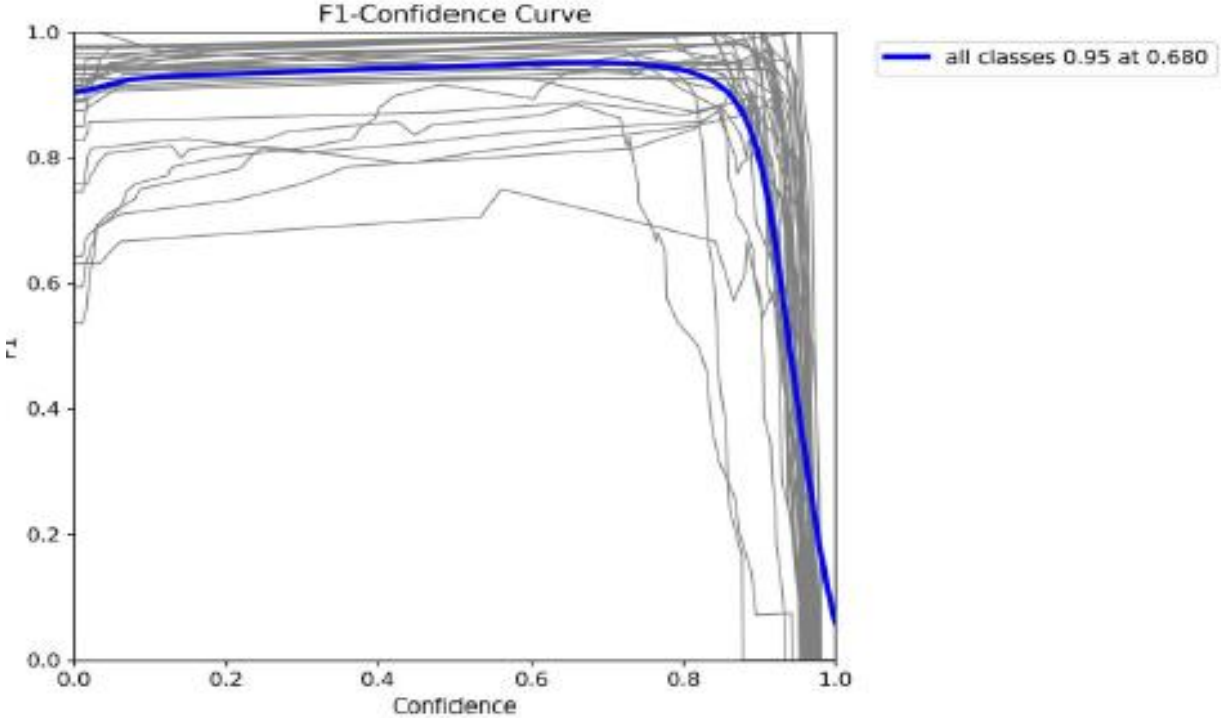


Figure 2: F1-Confidence Curve (0.95 at 0.680).

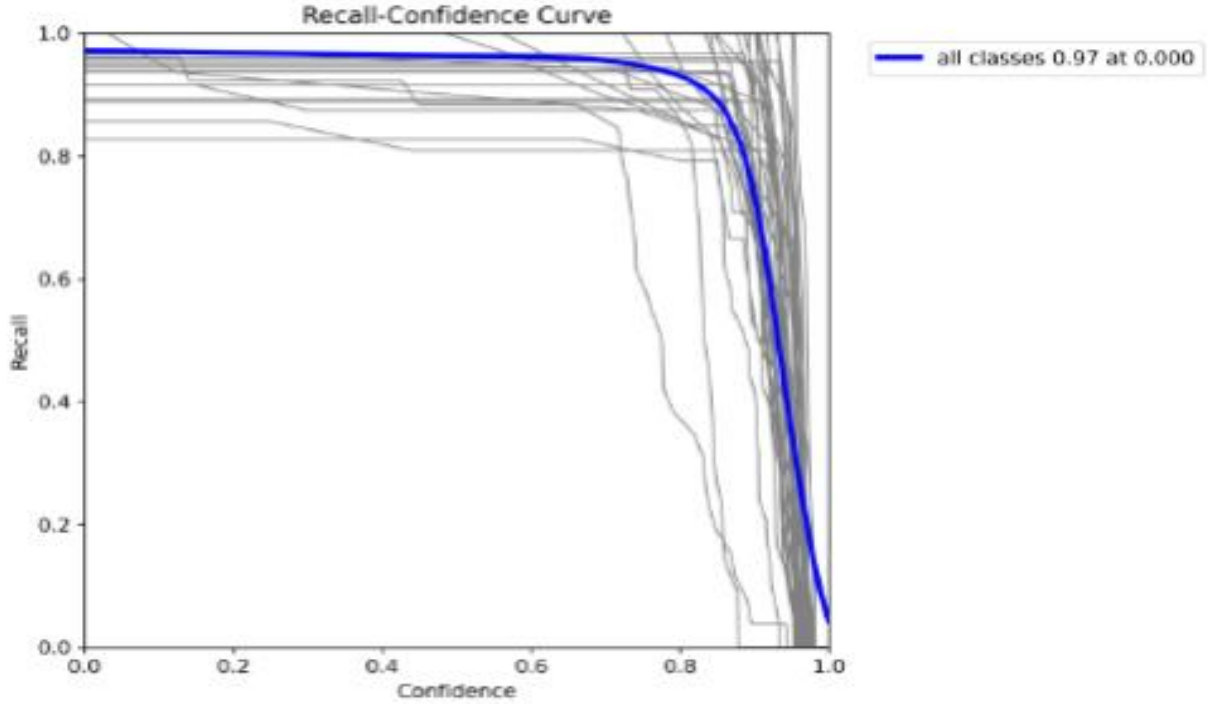



Figure 3: Recall-Confidence Curve (0.97 at 0.000)

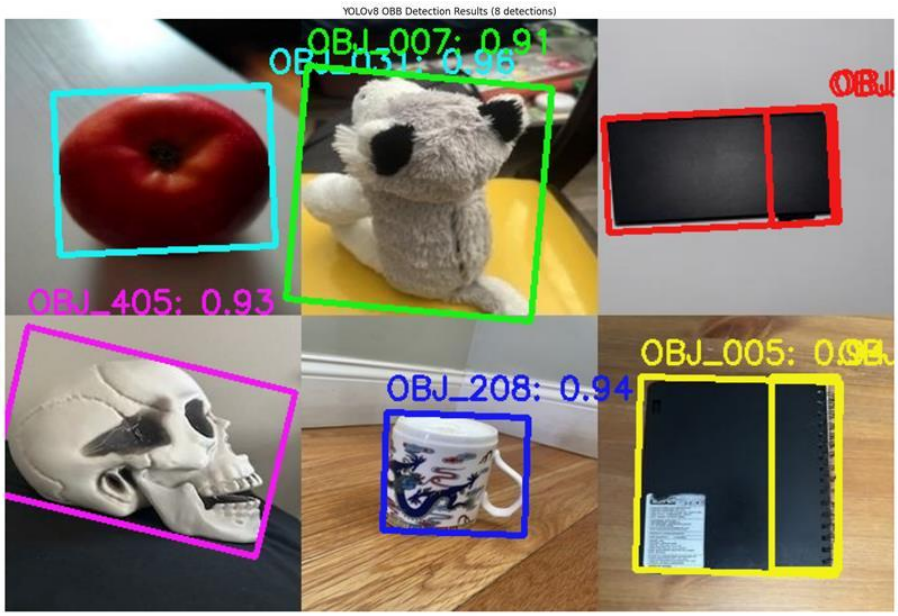
# Observations

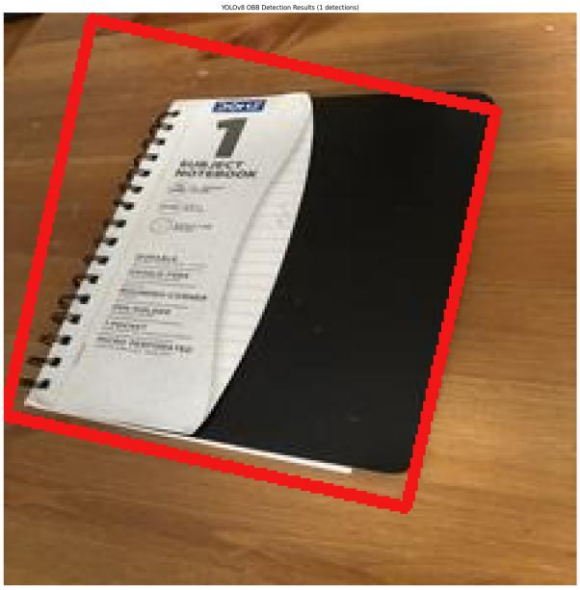
From the training logs initially, we could see the loss steadily dropping with each epoch while the mean average precision kept climbing, which was good the model was able to learn not just memorizing the data. By the time we hit epoch 80, the validation scores did not improve they reached a saturation point. At that point continuing to train would have been waste of GPU resources.

One thing we were concerned about early on was how the model would handle objects photographed at different angles. For this problem we stacked two layers of augmentation Roboflow built-in adjustments on top of YOLOv8 own Mosaic and Mixup techniques. Both together made a noticeable difference in how well the model generalized.

The other challenge was that the dataset we were given only had a training split and a test split no validation set. This matters because if you're tuning your model and checking performance against the same data you trained on. We pulled 20% out of the training data to use as validation, so every mean average precision score we reported was based on images the model had genuinely never seen during training. That gives us much more confidence that the results are real.

# Demo Results and Observations





Thanks to the training we facilitated for our model, it was able to accurately detect each unique item in all four photos. We were relieved that OBJ_021 and OBJ_006 were detected in the fourth image. The OBJ_021 folder had multiple different lotion bottles each with less training images than the other objects from the rest of the class. For OBJ_006, the hairclip, many of the training

images didn't have the hairclip standing out much from the background but our model could still notice the shape and structure of the hairclip. For the 2nd and 3rd demo, the model was able to accurately detect the black notebook and the Coca-Cola from what we saw in the output of the code cell before the cell that ran the inference but the label wasn't showing up. This could be because the bounding boxes were large enough that the labels were showing outside the photo.

# Conclusion

With the successful demo of our trained YOLOv8 model, we can say our project is ready to be deployed for automated student attendance tracking. The next steps would be setting up the tracking system, which would involve assigning the object IDs to the NUIDs or names of each student. Compared to training the model, these steps would be very easy to accomplish. The only improvements for the model's output would be making sure there's always a way for the bounding boxes to have their object IDs showing in the frame when the boxes are drawn.